ELET, the error of the network for this particular input, and MU, a constant set at the beginning of the program. To understand the need for ELET in this calculation, imagine that an image placed on the input produces a *small* error in the output signal. Next, imagine that another image applied to the input produces a *large* output error. When adjusting the weights, we want to nudge the network more for the second image than the first. If something is working poorly, we want to change it; if it is working well, we want to leave it alone. This is accomplished by changing each weight in proportion to the current error, ELET.

To understand how MU affects the system, recall the example of the paratrooper. Once he determines the downhill direction, he must decide how far to proceed before reevaluating the slope of the terrain. By making this distance short, one meter for example, he will be able to precisely follow the contours of the terrain and always be moving in an optimal direction. The problem is that he spends most of his time evaluating the slope, rather than actually moving down the hill. In comparison, he could choose the distance to be large, say 1000 meters. While this would allow the paratrooper to move rapidly along the terrain, he might overshoot the downhill path. Too large of a distance makes him jump all over the country-side without making the desired progress.

In the neural network, MU controls how much the weights are changed on each iteration. The value to use depends on the particular problem, being as low as 10^{-6}, or as high as 0.1. From the analogy of the paratrooper, it can be expected that too small of a value will cause the network to converge too slowly. In comparison, too large of a value will cause the convergence to be erratic, and will exhibit chaotic oscillation around the final solution. Unfortunately, the way neural networks react to various values of MU can be difficult to understand or predict. This makes it critical that the network error (i.e., ESUM) be monitored during the training, such as printing it to the video screen at the end of each iteration. If the system isn't converging properly, stop the program and try another value for MU.

Evaluating the Results

So, how does it work? The training program for vowel recognition was run three times using different random values for the initial weights. About one hour is required to complete the 800 iterations on a 100 MHz Pentium personnel computer. Figure 26-9 shows how the error of the network, ESUM, changes over this period. The gradual decline indicates that the network is learning the task, and that the weights reach a near optimal value after several hundred iterations. Each trial produces a different solution to the problem, with a different final performance. This is analogous to the paratrooper starting at different locations, and thereby ending up at the bottom of different valleys. Just as some valleys are deeper than others, some neural network solutions are better than others. This means that the learning algorithm should be run several times, with the best of the group taken as the final solution.